CHAPTER 14

Transactions, ACID & Isolation Levels

Part III · Data & Storage — keeping data correct under failure and concurrency.

LEARNING OBJECTIVES

- Explain what a transaction is and why operations are grouped into one unit.
- Explain each ACID property with a concrete example.
- Name the concurrency anomalies (dirty read, non-repeatable read, phantom, lost update).
- Compare the four isolation levels and the correctness-vs-performance dial.
- Explain, at a high level, how isolation is implemented (locking vs MVCC).
- Understand why transactions are hard across many machines.

REAL-WORLD ANALOGY — TRANSFERRING MONEY

Moving $30 from Alice's account to Bob's is really *two* steps: subtract from Alice, add to Bob. If the power fails after the first step, money has vanished into thin air. A **transaction** guarantees both steps happen together or neither does — the transfer is **all-or-nothing**. And if Alice and Bob both try to withdraw from a shared account at the same instant, transactions make sure the account can't be double-spent. This chapter is how databases keep data correct when things fail and when many people act at once.

14.1 The problem: failure and concurrency

Two things threaten correct data. First, **failure**: a multi-step operation can be interrupted halfway, leaving the data in a broken, partial state. Second, **concurrency**: when many users read and write the same data simultaneously, their operations can interleave in ways that corrupt it. **Transactions** are the database mechanism that defends against both — and they are the deepest reason relational databases are trusted with money and orders.

14.2 What a transaction is

A **transaction** groups several operations into a single, indivisible unit. You `BEGIN`, do the work, and either `COMMIT` (make it all permanent) or `ROLLBACK` (undo everything). It's all-or-nothing.

```
BEGIN;
  UPDATE accounts SET balance = balance - 30 WHERE id = 'alice';
  UPDATE accounts SET balance = balance + 30 WHERE id = 'bob';
COMMIT;   -- both updates happen, or (on error) ROLLBACK undoes both
```

14.3 ACID in depth

Property	Meaning, via the transfer
Atomicity	All steps or none. Alice is never debited without Bob being credited. (Achieved with undo/rollback logs.)

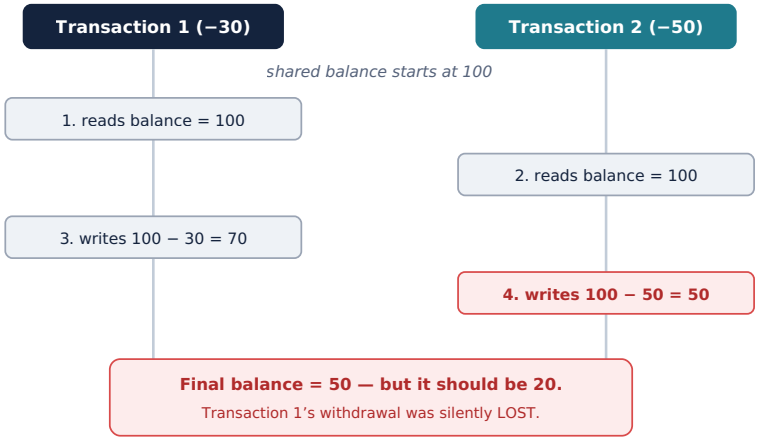
Consistency	The database moves from one valid state to another — all constraints still hold (no negative balance, totals add up).
Isolation	Concurrent transactions don’t corrupt each other; the result is as if they ran one at a time. (The subtle one — see isolation levels.)
Durability	Once committed, the data survives a crash or power loss. (Achieved by writing to a durable log on disk — recall Chapter 2 on volatility.)

14.4 What goes wrong without isolation

When transactions run concurrently without care, specific **anomalies** appear:

- **Dirty read:** you read data another transaction wrote but hasn’t committed — and it might roll back, so you read something that never really happened.
- **Non-repeatable read:** you read a row twice in one transaction and get different values, because another transaction changed it in between.
- **Phantom read:** you run the same query twice and new rows appear, because another transaction inserted them.
- **Lost update:** two transactions read the same value, each modifies it, and one silently overwrites the other.

The lost update is the classic. Watch two withdrawals race on a shared balance:



Both read the old value before either wrote. Proper **isolation** (or an atomic update) prevents this.

14.5 Isolation levels

Perfect isolation is expensive (it requires locking or extra work that limits how many transactions run at once). So databases offer **isolation levels** — a dial trading correctness for performance. Each level allows fewer anomalies but costs more concurrency:

Level	Dirty read	Non-repeatable	Phantom
Read Uncommitted	possible	possible	possible
Read Committed	prevented	possible	possible

Repeatable Read	prevented	prevented	possible*
Serializable	prevented	prevented	prevented

*Some databases also prevent phantoms at Repeatable Read. **Read Committed** is a common default (PostgreSQL, Oracle); **Repeatable Read** is MySQL/InnoDB's default; **Serializable** is the strictest and safest, and the slowest.

14.6 How isolation is implemented

- **Locking (pessimistic):** a transaction locks the rows it touches so others must wait. Safe but reduces concurrency and can cause **deadlocks** (two transactions each waiting on the other).
- **MVCC (multi-version concurrency control):** the database keeps multiple versions of a row, so readers see a consistent snapshot without blocking writers. Used by PostgreSQL and MySQL/InnoDB — it's why "readers don't block writers."
- **Optimistic concurrency:** assume conflicts are rare; don't lock, but check at commit whether anything changed, and retry if so. Great when contention is low.

14.7 Choosing a level — the trade-off

KEY INSIGHT — ISOLATION IS A CORRECTNESS/THROUGHPUT DIAL

Higher isolation means fewer anomalies but more locking and lower throughput. Most applications run at **Read Committed** or **Repeatable Read**. Reach for **Serializable** only where correctness is critical (money, inventory), and accept the performance cost. And keep transactions **short**: a long transaction holds locks and starves everyone else — a top cause of database contention.

14.8 Transactions across many machines

WHY THE DATA TIER IS HARD — AGAIN

ACID is (relatively) easy on **one** machine. Across many machines — the moment you replicate or shard (Chapters 16–17) — guaranteeing atomicity and isolation becomes genuinely hard. Distributed transactions (two-phase commit) are slow and fragile, so large systems often **give up strict ACID across services** in favor of the **Saga** pattern and **eventual consistency** (Chapter 38). This tension — strong transactional guarantees vs horizontal scale — is a central theme of Part IV.

Common mistakes

WATCH OUT

- Doing a multi-step, money-or-state change *without* wrapping it in a transaction.
- The **lost update** race — read-modify-write without proper isolation or an atomic update.
- Defaulting to **Serializable** everywhere and crippling throughput, or assuming the default level prevents all anomalies.

- Holding a transaction open for a long time (network calls inside it), causing lock contention and deadlocks.
- Expecting single-machine ACID guarantees to hold, unchanged, across a sharded or microservice system.

Interview questions

1. What is a transaction? Explain each letter of ACID with an example.
2. Describe dirty read, non-repeatable read, phantom read, and lost update.
3. Compare the four isolation levels. Which anomalies does each allow?
4. What is the trade-off as you raise the isolation level?
5. What is MVCC, and how does it differ from locking?
6. Why are transactions hard across multiple machines, and what do systems use instead?
7. Why should transactions be kept short?

Hands-on exercises

1. Write a money-transfer transaction and describe what a rollback would undo.
2. Reproduce a lost update by running two concurrent read-modify-write sessions, then fix it with an atomic update or higher isolation.
3. For each anomaly, name the lowest isolation level that prevents it.
4. Decide an isolation level for: a like button; an inventory-decrement at checkout. Justify each.
5. Explain why a transaction that makes an external API call in the middle is dangerous.

Mini project

BREAK IT, THEN FIX IT

Create an `accounts` table and open two database sessions. Deliberately reproduce a **lost update**: in both sessions read the same balance, then have both write a decrement — observe the final value is wrong. Now fix it two ways and compare: (1) an **atomic** update (`SET balance = balance - 30` in one statement), and (2) raising the **isolation level** to Serializable and retrying on conflict. Seeing the anomaly and both fixes with your own hands makes isolation concrete.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — WHERE TRANSACTIONS ARE NEEDED

Go through your schema and mark every operation that must be a **transaction**: creating a post and incrementing the author's post count together; a follow that updates two counters; a like that must not be double-counted. For the denormalized counters you added in Chapter 13, decide how transactions (or atomic increments) keep them consistent. Pick an **isolation level** for each critical path, reserving Serializable for

anything that must never be wrong. Note which of these get harder once the data is sharded (Chapter 17) — the setup for Part IV.

Chapter summary

- A **transaction** groups operations into an all-or-nothing unit (`BEGIN` … `COMMIT` / `ROLLBACK`), defending against failure and concurrency.
- **ACID** = Atomicity, Consistency, Isolation, Durability — the guarantees that make relational databases trustworthy.
- Without isolation, concurrency causes **dirty reads, non-repeatable reads, phantoms, and lost updates**.
- **Isolation levels** (Read Uncommitted → Serializable) dial correctness against performance; most apps use Read Committed or Repeatable Read.
- Isolation is implemented with **locking** or **MVCC** (readers don’t block writers); keep transactions **short**.
- Strong ACID is easy on one machine and **hard across many** — large systems often trade it for Sagas and eventual consistency (Part IV).

COMING NEXT

Chapter 15 — NoSQL Databases. We meet the alternative to the relational model: key-value, document, wide-column, and graph stores that trade JOINs and strict ACID for massive scale and flexibility — and learn exactly when each is the right tool.